



Semi-Structured Distributional Regression

David Rügamer, Chris Kolb & Nadja Klein

To cite this article: David Rügamer, Chris Kolb & Nadja Klein (2024) Semi-Structured Distributional Regression, The American Statistician, 78:1, 88-99, DOI: [10.1080/00031305.2022.2164054](https://doi.org/10.1080/00031305.2022.2164054)

To link to this article: <https://doi.org/10.1080/00031305.2022.2164054>



View supplementary material [↗](#)



Published online: 10 Feb 2023.



Submit your article to this journal [↗](#)



Article views: 1257



View related articles [↗](#)



View Crossmark data [↗](#)



Citing articles: 13 View citing articles [↗](#)

Semi-Structured Distributional Regression

David Rügamer^{a,b}, Chris Kolb^{a,b}, and Nadja Klein^{c,d}

^aDepartment of Statistics, LMU Munich, Munich, Germany; ^bMunich Center for Machine Learning, Munich, Germany; ^cChair of Statistics and Data Science, Humboldt-Universität zu Berlin, Berlin, Germany; ^dResearch Center Trustworthy Data Science (UA Ruhr) and Department of Statistics (Technische Universität Dortmund), Chair of Uncertainty Quantification and Statistical Learning, Dortmund, Germany

ABSTRACT

Combining additive models and neural networks allows to broaden the scope of statistical regression and extend deep learning-based approaches by interpretable structured additive predictors at the same time. Existing attempts uniting the two modeling approaches are, however, limited to very specific combinations and, more importantly, involve an identifiability issue. As a consequence, interpretability and stable estimation are typically lost. We propose a general framework to combine structured regression models and deep neural networks into a unifying network architecture. To overcome the inherent identifiability issues between different model parts, we construct an orthogonalization cell that projects the deep neural network into the orthogonal complement of the statistical model predictor. This enables proper estimation of structured model parts and thereby interpretability. We demonstrate the framework's efficacy in numerical experiments and illustrate its special merits in benchmarks and real-world applications.

ARTICLE HISTORY

Received July 2022
Accepted December 2022

KEYWORDS

Generalized additive models for location; Scale and shape; Identifiability; Multimodal learning; Neural networks; Orthogonalization

1. Introduction

In many applications, it is crucial to capture uncertainties of predictions besides mere point estimates. In machine learning, a distinction is made between aleatoric uncertainty—uncertainty that is irreducible, that is, that stems from the data-generating process—and epistemic uncertainty—uncertainty that relates to the imperfect epistemic state of the system or modeler. This includes the practical limitations researchers face in knowledge discovery, for example, insufficient sample sizes, unobserved variables, measurement error, and model misspecification. Under correct model specification, epistemic uncertainty vanishes in the large-sample limit (Hüllermeier and Waegeman 2021). In statistical regression, these two concepts are usually addressed by distributional modeling approaches attempting to model the aleatoric uncertainty explicitly, and statistical inference, which provides statements about the regression coefficient uncertainty. Note that statistical inference usually only aims to address model uncertainty, not necessarily other hard-to-tackle types of epistemic uncertainty.

In deep learning, most approaches focus on epistemic uncertainty. Common approaches include variational inference (Blei, Kucukelbir, and McAuliffe 2017), stochastic weight averaging (Maddox et al. 2019) using first moments of stochastic gradient descent iterates together with a Gaussian low-rank assumption, or various forms of Laplace approximation (see, e.g., Daxberger et al. 2021) based on the idea of local linearization of neural networks (Immer, Korzepa, and Bauer 2021). One of the most successful and commonly used approaches is deep ensembles (Lakshminarayanan, Pritzel, and Blundell 2017). Deep ensembles

form a predictive uncertainty interval by constructing a mixture distribution from a small number (usually in the range of 5–10) of trained neural networks that have been initialized with different starting values. Their success is linked to the fact that multiple runs of an iterative, stochastic optimization routine learn different models each time, with predictions in regions of low uncertainty in the input space having a low variance, and predictions in regions where the model is uncertain exhibiting larger variance.

In contrast, our article focuses on modeling the aleatoric uncertainty more realistically by capturing the entire data-generating process of the outputs (rather than just average predictions of it) in a probabilistic manner. Similar approaches exist in deep learning, for example, mixture density networks (Bishop 1994), which explicitly model components of a mixture distribution using a neural network. These mixtures then form a flexible distributional model for the output which compared to our approach, however, lacks interpretability. In the special case of binary regression or classification tasks, a common approach is to use entropy-based measures from information theory (Hüllermeier and Waegeman 2021). Distributional regression also plays a role in reinforcement learning (Dabney et al. 2018) as a technique to estimate the distribution of returns for a given state-action pair, rather than just estimating the expected return. This can provide additional information about the uncertainty of the estimates, which can be used to improve the learning algorithm's performance.

In statistics, there has been a long-standing interest in modeling distributional quantities beyond the mean. An early example is given by quantile regression models (Koenker and Bassett Jr

1978; Koenker 2005). Another strand of literature started with the introduction of the well-known generalized additive model for location, scale and shape by Rigby and Stasinopoulos (2005). This model class not only models the conditional mean of a response as a function of covariates (treating other parameters such as the variance as nuisance parameters) but defines a distinct predictor function for each distributional parameter. This enables the prediction of entire conditional distributions instead of only the mean. Generalized additive models (Hastie and Tibshirani 1986) are sometimes referred to as structured additive regression models to reflect developments in the predictor structure to include, among others, random effects, varying coefficient models, Gaussian Markov random fields, or multi-variate tensor product interaction surfaces. Similarly, we refer to Rigby and Stasinopoulos (2005) and subsequently developed extensions more broadly as structured additive distributional regression (SADR; see, e.g., Rigby and Stasinopoulos 2005; Klein et al. 2015), instead of the usual abbreviation GAMLSS, to address the whole family of related models instead of a specific one. SADR explicitly models all distributional parameters of an arbitrary parametric distribution $\mathcal{D}$ as flexible functions of input features and thereby allows learning the whole conditional distribution.

1.1. Mean and Distributional Regression Models

Among the approaches that directly model predictive distributions, distributional regression can be considered a natural extension of traditional statistical regression models focusing on the mean. In particular, mean regression is used to estimate the expectation $\mu := \mathbb{E}_{\mathcal{D}}(Y|\mathbf{v})$ of a target variable Y using tabular input features $\mathbf{v}$. The most widespread mean regression is commonly known as the generalized linear model Nelder and Wedderburn (1972), defining the set of candidate models as

$$\mu = \mathbb{E}_{\mathcal{D}}(Y|\mathbf{v}) = h(\mathbf{v}^{\top} \mathbf{w})$$

with model weights or coefficients $\mathbf{w}$ and response function h . As an extension, generalized additive models Wood (2017) allow for general structured additive predictors

$$\mu = h(\sum_j f_j(\mathbf{v}))$$

for estimating functional (e.g., nonlinear, spatial) relationships between features and the response through generic function space representations f_j . Generalized linear models and generalized additive models usually assume only very few and low-dimensional interactions between features $\mathbf{v}$ in the additive predictor $\sum_j f_j(\mathbf{v})$. Due to the additivity and function space assumptions of feature effects, this so-called structured predictor allows for a straightforward interpretation of the model components. For example, in a generalized additive model with no interactions,

$$\mu = h(\sum_j f_j(v_j)),$$

each effect $f_j(v_j)$ is a function of feature v_j and the contribution on $h^{-1}(\mu)$ can be quantified explicitly by holding all other effects in the additive predictor constant. SADR extends generalized additive models by combining the idea of distributional regression and structured predictors. SADR thereby relates various,

potentially different additive predictors to each distributional parameter of an arbitrary parametric distribution. This allows the practitioner to explicitly model the aleatoric uncertainty of the data-generating processes. As in generalized additive models, the interpretation of feature effects on each of the distributional parameters, for example, the nonlinear effect of one feature on the variance of a normal distribution, is straightforward. Yet, structured additive models are limited in their application (mainly tabular data) and their simplistic model structure (e.g., not allowing higher-order interactions).

In this work, we extend SADR to a more flexible approach which we refer to as semi-structured distributional regression (SSDR). This approach combines the structured additive predictor with an unstructured model part learned through a deep neural network (DNN). Typically, the structured model part processes tabular feature information whereas the unstructured part can in addition also take other data formats such as images or texts into account. Our goal is a distributional modeling framework that covers classical deep learning applications, use cases requiring higher-order feature interactions or multimodal learning settings, while still preserving the interpretability of (structured parts of) the model. This framework can describe the entire conditional distribution of the response rather than only the mean (and thus point predictions), but also includes mean regression as a special case.

1.2. Related Work

Existing approaches in deep learning that fuse structured predictors for tabular data with other data modalities in a multimodal network are sometimes referred to as wide and deep neural networks. Several approaches exist (see, e.g., Cheng et al. 2016; Pölsterl et al. 2020, for two recent applications), but these wide and deep models only focus on mean regression and linear effects. Our proposal can also be seen as an extension of mean regression wide and deep models to distributional wide and deep networks.

Existing SADR models in statistics (e.g., Klein et al. 2015; Groll et al. 2019) assume purely structured additive predictors of available tabular input data. Several authors have described amalgamations of a simpler statistical model and a neural network. Sarle (1994) describes the commonalities and differences between statistical models and neural networks, De Waal and Du Toit (2011) and Brás-Geraldes, Papoila, and Xufre (2019) consider generalized additive models when framed as a neural network. Agarwal et al. (2021) propose learning the nonlinear additive functions of generalized additive models within a neural network using separate networks for each feature. Various authors in the statistical literature have also proposed a combination of statistical regression and neural networks. Tran et al. (2020) propose the class of deep generalized linear models, where only the conditional expectation μ is modeled through a predictor that is based on a DNN. Their model is similar to the deep Bayesian regression model of Hubin, Storvik, and Frommlet (2018) who implement exact Bayesian inference, while Tran et al. (2020) consider an approximation via fixed form variational Bayes and allow for additional random effects. In Umlauf, Klein, and Zeileis (2018) a single-layer neural network is included in a SADR model for a very specific choice

of weight and bias generation, but this approach does not allow for more complex or deep network structures. Existing wide and deep neural networks only focus on modeling the distribution mean (e.g., Cheng et al. 2016; Chen et al. 2018) or the hazard in survival regression (Pölsterl et al. 2020). Li, Reich, and Bondell (2021) propose a deep distributional regression approach by transforming the estimation problem into a constrained multi-class classification problem but do not use (semi-)structured additive predictors.

1.3. Simplified Problem Formulation

While approaches marrying statistical regression and deep learning exist, these methods either do not provide aleatoric uncertainty or are limited to special use cases. More importantly, none of the existing approaches that try to leverage interpretable regression models and deep learning actually preserve the interpretability of the structured regression part satisfactorily. This is due to an identifiability problem between the structured and unstructured predictors. For instance, consider a deep generalized linear model that fuses a DNN d with inputs $\mathbf{v}$ additively with a linear model predictor using the same features:

$$\eta = h^{-1}(\mu) = \mathbf{v}^\top \mathbf{w} + d(\mathbf{v}).$$

From the universal approximation theorem (Cybenko 1989) and related literature, it is well known that d can potentially approximate any continuous function under certain conditions. Thus, without loss of generality, assume that $d(\mathbf{v}) = \mathbf{v}^\top \mathbf{m} + f(\mathbf{v})$, decoupling the learned effect of d into a linear predictor part $\mathbf{v}^\top \mathbf{m}$ and some nonlinear effect f of $\mathbf{v}$. We can directly observe that the deep generalized linear model encompasses an identifiability issue, as we can arbitrarily shift a linear portion of $\mathbf{v}$ from the linear model part in η to the deep part d of the model and vice versa, for example:

$$\mathbf{v}^\top \mathbf{w} + d(\mathbf{v}) = \mathbf{v}^\top \mathbf{w} + \mathbf{v}^\top \mathbf{m} + f(\mathbf{v}) = \mathbf{v}^\top \mathbf{m} + \tilde{d}(\mathbf{v}).$$

While not relevant for prediction, this identifiability issue inhibits interpretability of the model. In the deep generalized linear model example, it is unclear how much of the linear effect the model will be attributed to the structured predictor and how much to the deep model part. This problem becomes even more entangled for more complex structured predictors as commonly used in generalized additive models or SADR.

1.4. Main Contributions

We present a novel neural network-based framework for the combination of SADR and (deep) neural networks. We address challenges with estimation and tuning of such a model, and in particular, propose a solution to the inherent identifiability issue in this model class based on a well-chosen orthogonalization cell. This cell permits the joint modeling of the structured model part and the unstructured DNN predictor in a unifying, end-to-end trainable network, while preserving the interpretability of the structured part. On the one hand, our model class subsumes classical statistical regression models such as generalized additive models or SADR as special cases within a neural network architecture while having similar or even superior estimation performance. On the other hand, due to the generality of our

approach, extensions of existing regression approaches can be built in a straightforward fashion. This facilitates, for example, combining interpretable structured effects of tabular features with a DNN to capture potential higher-order interactions or multimodal learning problems in which multiple different data modalities such as tabular, image, or text data are present (see, e.g., Section 6). The optimization using deep learning platforms and first-order methods further fosters the applicability in settings with large amounts of data (both observations and covariates) as, for example, demonstrated in Rügamer et al. (2022).

The rest of the article is as follows. We start with a background section on SADR and introduce our notation in Section 2. After presenting semi-structured distributional regression and addressing the accompanied identifiability issue in Section 3, we describe implementation details in Section 4. In Section 5, we first examine how to achieve the identifiability of our model. Thereafter, we investigate its estimation and prediction performance in comparison with state-of-the-art distributional models. We then apply our method to several benchmark datasets in Section 6. Finally, we will use the proposed framework in a multimodal data setting to predict prices of rental apartments in Section 7. We implement our proposed framework in a corresponding software package, available at <https://github.com/davidruegamer/deepregression>. All codes to reproduce results from numerical experiments, benchmarks and our application are available at https://github.com/davidruegamer/semi-structured_distributional_regression.

2. Structured Additive Distributional Regression

In this section, we introduce SADR more formally and define the basic notation. For more details, see, for example, Klein, Kneib, and Lang (2015).

2.1. Distributional Regression and Notation

As briefly introduced before, SADR models aim at estimating arbitrary parametric distributions $\mathcal{D}(\theta_1, \dots, \theta_K)$ by learning the corresponding distributional parameters $\boldsymbol{\theta} = (\theta_1, \dots, \theta_K)^\top \in \Theta \subseteq \mathbb{R}^K$. SADR allows to regress features $\mathbf{v}$ on potentially all parameters θ_k , $k = 1, \dots, K$ of the response distribution $\mathcal{D}$. In the spirit of generalized additive models, each of the K distributional parameters is related to (possibly different subsets of) available features $\mathbf{v}$ through a monotonic and differentiable response function

$$\theta_k(\mathbf{v}) = h_k(\eta_k(\mathbf{v})).$$

The predictors $\eta_k(\mathbf{v}) \in \mathbb{R}$ specify the relationship between features $\mathbf{v}$ and the (transformed) parameters $h_k^{-1}(\theta_k)$, while h_k ensures that possible parameter space restrictions on θ_k are fulfilled, for example, $h_k(\cdot) = \exp(\cdot)$ to ensure positivity for a variance parameter. Note that $\mathcal{D}$ itself can be also a more complicated distribution, for example, a mixture of distributions.

2.2. Predictor Structure

The structured predictors η_k in SADR are assumed to be an additive decomposition of structured linear parts $f_{k,0}(\mathbf{x}) = b_k + \mathbf{x}^\top \mathbf{w}_k$ and structured nonlinear functions $f_{k,j}(z_j)$:

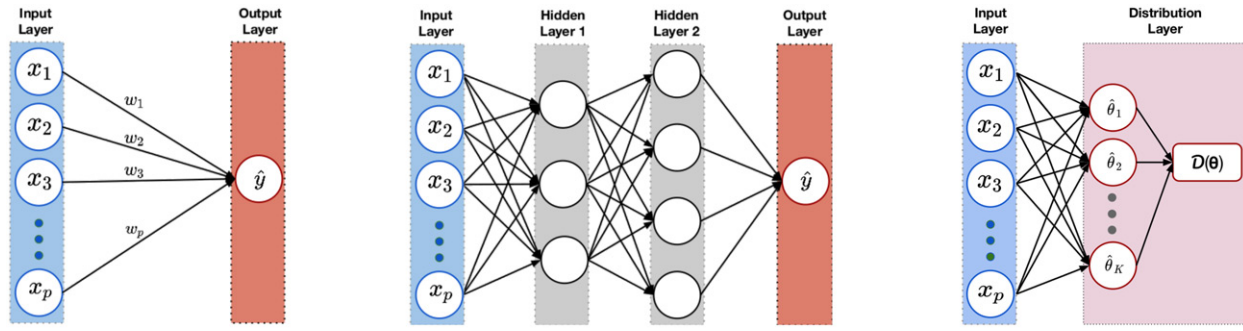


Figure 1. Visualization of a single neuron representing a linear model (left), an exemplary multi-layer perceptron (center), and a simple network with a distributional layer as output layer (right).

$$\eta_k = f_{k,0}(\mathbf{x}) + \sum_{j=1}^{r_k} f_{k,j}(z_j). \quad (1)$$

Note that we have suppressed the index k in the subset of feature inputs to not overload notation. However, SADR allows specifying all additive terms individually for all parameters θ_k . This can be relevant for example when prior information is available about which features are relevant for the location or the scale of the response.

The functions $f_{k,j}(\cdot)$ represent penalized smooth nonlinear effects of univariate or low-dimensional features using a linear combination of $L_{k,j}$ appropriate basis functions. A univariate nonlinear effect of feature z_j is, for example, approximated by $f_{k,j}(z_j) \approx \sum_{l=1}^{L_{k,j}} B_{k,j,l}(z_j) w_{k,j,l}$, where $B_{k,j,l}(z_j)$ is the l th basis function (such as regression splines, polynomial bases or B-splines) evaluated at z_j . Tensor product representations allow for two- or moderate-dimensional nonlinear interactions of a subset of $\mathbf{z}$. It is also possible to represent discrete spatial information or cluster-specific effects in this way (see, e.g., Wood 2017). This representation also allows for random effects in the additive predictor, as these can be represented by ridge-penalized linear effects (see Section 4.1 for details).

3. Semi-Structured Distributional Regression

Our proposed framework advances SADR by extending the additive predictors η_k to include one or more latent representations learned through (possibly different) DNNs. To this end, we embed SADR into a neural network and learn the distribution by adapting deep probabilistic modeling approaches.

3.1. Embedding Regression Models in a Neural Network

From a statistical perspective, a neural network with an input layer and an output layer with a single neuron, as depicted in Figure 1 (left), can be viewed as equivalent to a linear regression model. Just as a linear regression model takes input features $\mathbf{x}$ and learns a set of weights $\mathbf{w} = (w_1, \dots, w_p)^\top$ that can be used to make predictions $\hat{y} = \mathbf{x}^\top \mathbf{w}$, the output neuron takes inputs $\mathbf{x}$, applies weights $\mathbf{w}$ and produces an activation through a linear combination $\mathbf{x}^\top \mathbf{w}$ of the inputs and weights. Finally, an output activation function h^{out} is applied to the activation to compute the neuron's output. In the case of linear regression, the output activation function is simply the identity function $h^{\text{out}}(x) = x$. In statistical terminology, the activation corresponds to the lin-

ear predictor and the output activation function to the response function in generalized linear models. For example, if a neural network without hidden units and a single neuron in its output layer uses a sigmoid activation function, that is, $h^{\text{out}}(x) = 1/(1 + e^{-x})$, it can be considered equivalent to a logistic regression model.

In slightly more complex “shallow” networks using a single hidden layer with M neurons and a single output neuron, the model's output is given by the transformation $\hat{y} = h^{\text{out}}(b^{\text{out}} + \sum_{m=1}^M w_m^{\text{out}} h(\mathbf{x}^\top \mathbf{w}_m + b_m))$ of an input $\mathbf{x}$, where $\mathbf{w}_m$ denotes the set of weights associated with the m th hidden unit, b_m is a so-called bias term for the hidden layer, w_m^{out} and b^{out} the weights and bias of the output layer, and h the activation function of the hidden layer. The bias terms can be seen as intercepts added for the same purpose (to account for a mean shift in the outcome values). Multi-layer perceptrons, one of the most common neural networks, follow the same principle, but also allow for more than one hidden layer of interconnected neurons (see, e.g., Figure 1, center, for a 2-hidden layer perceptron). Multi-layer perceptrons are powerful models that can be used to solve a wide range of problems and can learn to approximate any continuous function to a certain degree of accuracy (Hornik, Stinchcombe, and White 1989). In supplementary material F, we describe further types of neural networks, their purpose, and typical fields of application.

From a computational point of view, neural networks are attractive since common deep learning platforms scale well to large amounts of data. Their optimization, based on computationally cheap first-order methods, avoids the costly computation of the Hessian and can be automated without the need to manually derive model gradients. Neural networks are trained by minimizing a loss function, which needs to be differentiable. As this is the case for the negative log-likelihood function of commonly encountered distributions, training neural networks is often equivalent to maximum likelihood estimation.

3.2. Embedding SSDR in a Neural Network

Similar to the implementation of simpler regression models in neural networks, SADR and our extension SSDR can be implemented in a neural network to leverage the scalability and flexibility of deep learning software. This can be done by creating K subnetworks for every parameter θ_k , $k = 1, \dots, K$, which in turn can have multiple subnetworks themselves to define the additive predictor structure (see Section 3.2.3).

3.2.1. Output Model Structure

In order to implement SADR in a neural network, we define the last layer of the network as a distributional layer (see Figure 1, right), a layer with K neurons that computes the predicted distribution $\mathcal{D}(\theta_1(\mathbf{v}), \dots, \theta_K(\mathbf{v}))$ based on the outputs $\eta_k(\mathbf{v})$ of the subnetworks for the $\theta_k, k = 1, \dots, K$. Given a realization y of Y , the model can be estimated by optimizing the negative log-likelihood $-\ell(\theta) = -\log p_{\mathcal{D}}(y|\theta_1(\mathbf{v}), \dots, \theta_K(\mathbf{v}))$ based on the probability density or mass function $p_{\mathcal{D}}$ of $\mathcal{D}$ evaluated at the estimated parameters $\hat{\theta}$. For simplicity, we consider a univariate response Y , but the generalization to multivariate responses is readily available.

3.2.2. Network Inputs

The subnetworks for the distributional parameters each process the feature vector $\mathbf{v}$ (or different subsets of it). We consider $\mathbf{v}$ to be the concatenated set of input features $\mathbf{x} = (x_1, \dots, x_p) \in \mathbb{R}^p$ modeled as structured linear effects, input features $\mathbf{z} = (z_1, \dots, z_r) \in \mathbb{R}^r$ modeled as structured nonlinear effects, and features $\mathbf{u} = (u_1, \dots, u_q) \in \mathbb{R}^q$ that are passed through a DNN and constitute the unstructured model part in one or more of the additive predictor(s) $\eta_k, k = 1, \dots, K$.

3.2.3. Predictor Structure

Based on the additive predictor definition of SADR in (1), we define the semi-structured predictors η_k as additive decomposition of structured linear parts, structured nonlinear functions, and one or more unstructured predictors $d_{k,j}(\mathbf{u})$ constituting linear combinations of latent features learned through DNNs. The inputs for these DNNs can be subsets of the features $\mathbf{u}$ and can also be (partially) identical to the features $\mathbf{x}, \mathbf{z}$. The semi-structured predictors are given by

$$\eta_k = f_{k,0}(\mathbf{x}) + \sum_{j=1}^{r_k} f_{k,j}(z_j) + \sum_{j=1}^{g_k} d_{k,j}(\mathbf{u}).$$

Again, we have suppressed the index k in the subset of feature inputs. We further assume that the DNN predictors can be represented as $d_{k,j}(\mathbf{u}) = \hat{\mathbf{u}}_{k,j}^\top \boldsymbol{\gamma}_{k,j}$, with latent features $\hat{\mathbf{u}}_{k,j}$ taken as the outputs of the network trunk (all DNN layers up to the penultimate layer) and $\boldsymbol{\gamma}_{k,j}$ being the last-layer weights forming the network head (see Figure 2).

In contrast to a standard DNN, our approach makes structured effects explicit which can help in situations where this structured model assumption holds. When using a large purely unstructured network, an extensive (hyper-)parameter search is required to find the right amount of regularization in order to effectively model these simpler effects. A semi-structured model, in turn, includes the structured model as a special case for large penalization but can work better than a structured model (through the additional DNN part) if the structured model is misspecified. In other words, our approach does not only provide an extension of structured models to include non-tabular data modalities but also offers the flexibility to account for various data-generating processes.

3.3. Identifiability

Identifiability is crucial when features overlap in the structured and unstructured model parts. More formally, we here address

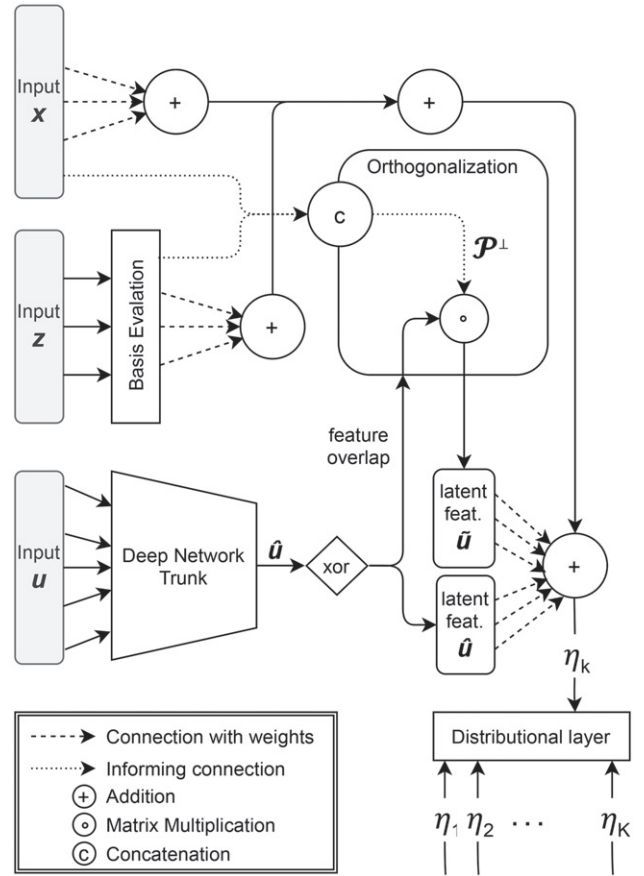


Figure 2. Exemplary SSQR architecture incorporating an orthogonalization cell: Structured linear features $\mathbf{x}$ and structured nonlinear features $\mathbf{z}$ (represented by their respective basis evaluations) are fed into the cell, concatenated and combined with the DNN using the orthogonalization operation. The solid lines represent the informing connections that either simply combine information or create a projection matrix $\mathcal{P}^\perp$. The latter is multiplied with the latent features $\hat{\mathbf{u}}$ to disentangle the structured and unstructured parts. The final additive predictors η_k are created by adding the structured predictors to the (orthogonalized) linear combination of latent features $\hat{\mathbf{u}}$ and their weights before being passed on to the distributional layer.

the following identifiability issue for distributional regression with structured additive predictor(s) η_k^{str} and unstructured additive predictor(s) $\eta_k^{\text{unstr}}, k = 1, \dots, K$:

Definition 3.1 (Semi-structured identifiability). We say that a semi-structured distributional regression is identified in its structured model parts if there exists no

$$\begin{aligned} \xi_k \neq 0 : \eta_k &= \eta_k^{\text{str}} + \eta_k^{\text{unstr}} = (\eta_k^{\text{str}} - \xi_k) + (\eta_k^{\text{unstr}} + \xi_k) \\ &= \check{\eta}_k^{\text{str}} + \check{\eta}_k^{\text{unstr}} = \check{\eta}_k, \end{aligned}$$

where $\ell(\check{\theta}) = \ell(\theta)$ and $\theta_k(\eta_k)$ in $\check{\theta}$ is replaced by $\theta_k(\check{\eta}_k)$ for any $k \in \{1, \dots, K\}$.

To derive a theoretical concept assuring identifiability in the following, we assume without loss of generality that we are only interested in linear effects of pre-specified features $\mathbf{x}$ and allow for a single additional DNN predictor d_k with arbitrary features $\mathbf{u}$ in η_k for one distributional parameter θ_k . The treatment of the more general case can be found in supplementary material A. We again suppress the index k in the feature vectors and design matrices for readability. As we will explain in a later remark, we consider identifiability of every additive predictor η_k on the level

of n observations. Therefore, let $(\mathbf{x}_1^\top, \dots, \mathbf{x}_n^\top)^\top =: \mathbf{X} \in \mathbb{R}^{n \times p}$, $1 \leq p \leq n$, be the structured feature matrix for $n \in \mathbb{N}$ observations and $\mathbf{w}_k \in \mathbb{R}^p$ the corresponding weights. Further, define the collection of n feature vectors $(\mathbf{u}_1^\top, \dots, \mathbf{u}_n^\top)^\top$ as $\mathbf{U} \in \mathbb{R}^{n \times q}$, which is fed into the DNN d_k , and let $\hat{\mathbf{U}}_k \in \mathbb{R}^{n \times s}$, denote the matrix collecting n latent feature vectors of length $s, s \leq n$, obtained from the second-last layer of d_k . Finally, let $\boldsymbol{\eta}_k = (\eta_{k,1}, \dots, \eta_{k,n})^\top \in \mathbb{R}^n$ be the final predictor vector.

If not constrained, d_k is able to capture linear effects of any of its features $\mathbf{U}$, including those also present in $\mathbf{X}$, thus making the attribution of shared effects to either one of the model parts in $\boldsymbol{\eta}_k$ not identifiable. The following Lemma shows how the orthogonalization induces a meaningful decomposition of learned effects in $\boldsymbol{\eta}_k$ and guarantees identifiability in the sense of Definition 3.1.

Lemma 3.2 (Orthogonalization). Let $\mathcal{P}_X \in \mathbb{R}^{n \times n}$ the projection matrix for which $\mathcal{P}_X \mathbf{A}$ is the linear projection of $\mathbf{A} \in \mathbb{R}^{n \times s}$, $s \leq n$, onto the column space spanned by the features of $\mathbf{X}$ and $\mathcal{P}_X^\perp := \mathbf{I}_n - \mathcal{P}_X$ the projection into the respective orthogonal complement. Then, replacing the latent features $\hat{\mathbf{U}}_k$ with

$$\tilde{\mathbf{U}}_k = \mathcal{P}_X^\perp \hat{\mathbf{U}}_k$$

and multiplying the result with the last layer's weights $\boldsymbol{\gamma}_k \in \mathbb{R}^s$, ensures a decomposition of the final predictor

$$\boldsymbol{\eta}_k = \mathbf{X} \mathbf{w}_k + \tilde{\mathbf{U}}_k \boldsymbol{\gamma}_k \quad (2)$$

into an identified linear part learned from features $\mathbf{X}$ and a nonlinear part learned from features $\tilde{\mathbf{U}}_k$.

A proof of Lemma 3.2 can be found in the supplementary material A and can be used to proof the identifiability of structured terms in $\boldsymbol{\eta}_k$ as follows.

Theorem 3.3 (Identifiability). Replacing $\hat{\mathbf{U}}_k$ with $\tilde{\mathbf{U}}_k$ and multiplying the result with the weights $\boldsymbol{\gamma}_k$ ensures identifiability of the structured linear part in the final predictor (2).

A proof is given in supplementary material A.

Remark 3.4. If $\mathbf{x}$, $\mathbf{z}$, and $\mathbf{u}$ overlap in their features, we first reparameterize the structured nonlinear model part to ensure identifiability between the structured linear and nonlinear parts and then combine the two into one joint predictor (not shown in Figure 2). The nonlinear part can then be interpreted as the nonlinear deviation from the corresponding linear effect. Finally, we apply the orthogonalization using the joint structured predictor to separate it from the unstructured DNN predictor.

Remark 3.5. When the DNN and the structured part share p columns with $p > n$, $\mathcal{P}_X^\perp$ is equal to a zero matrix $\mathbf{O}_{n \times n}$, making $\hat{\mathbf{U}}_k$ de facto irrelevant. In this case the estimated model is equivalent to the defined structured model. We see this edge case rather as a property than a limitation of our framework as the key requirement of the proposed architecture is to provide identifiable structured effects, which in this case is only possible by excluding the unstructured predictor part.

Remark 3.6. Last, we note that an alternative orthogonalization type and commonly used technique to make effects identifiable

in structured models (see, e.g., Rügamer et al. 2018) is to calculate $\boldsymbol{\Xi}_k$, a matrix with columns spanned by $\ker(\hat{\mathbf{U}}_k^\top \mathbf{X})$, and then use $\tilde{\mathbf{U}}_k = \hat{\mathbf{U}}_k \boldsymbol{\Xi}_k$. This would have the advantage of being applied on the columns of $\hat{\mathbf{U}}_k$ and not its rows and would therefore make the orthogonalization independent of the sample or batch size. However, as $\hat{\mathbf{U}}_k$ is updated in each iteration, the required null space $\ker(\hat{\mathbf{U}}_k^\top \mathbf{X})$ is also changing and can thus not be used in an end-to-end differentiable architecture (as gradients cannot be calculated for a random null space). This is the reason we chose the orthogonalization as proposed in Lemma 3.2 and consider identifiability on the level of observations.

3.4. Unifying Network Architecture

In contrast to most previous approaches that combine structured regression and DNNs, we propose a unifying network that learns the structured effects of $\mathbf{x}$ and $\mathbf{z}$ using single unit hidden layers with linear activation functions and different regularization terms for each input type and each distributional parameter. The DNN model part(s) processing $\mathbf{u}$ can be arbitrarily specified to, for example, incorporate complex feature interactions. To ensure identifiability, we propose an orthogonalization cell based on the previous theorem. This allows us to combine structured and unstructured feature effects in a distributional regression setting while ensuring identifiability. Figure 2 and its caption provide a detailed explanation.

Remark 3.7. For latent features $\hat{\mathbf{u}}_{k,j}$ learned in DNNs $d_{k,j}$, a distinction is made between DNNs whose inputs are also part of the structured inputs $\mathbf{x}$, $\mathbf{z}$ and DNNs whose inputs only appear in the unstructured predictor (xor-node in Figure 2). In the latter case, the DNN outputs are directly summed up as a weighted combination with structured predictors (lower path after xor-node in Figure 2) and fed into the distributional layer. For those $\hat{\mathbf{u}}_{k,j}$ whose DNNs $d_{k,j}$ also share inputs with $\mathbf{x}$ or $\mathbf{z}$, the orthogonalization operation is applied before adding its outputs $\hat{\mathbf{u}}_{k,j}$ as weighted sums to the remaining predictor parts. While this approach ensures the identifiability of the structured model part, a custom orthogonalization for specific (nonoverlapping) inputs in $\mathbf{u}$ might also be interesting in applications (see Section 8).

4. Implementation Details

We now provide further details on various implementation aspects of the SSDR framework.

4.1. Penalization, Optimization, and Tuning

It is common ground that gradient descent routines used to train neural networks hold an implicit regularization behavior (see, e.g., Arora et al. 2019). However, in most of our experiments we observe rather coarse estimated nonlinear effects or even convergence difficulties when not penalizing structured nonlinear effects. We therefore allow for additional quadratic penalization of structured nonlinear effects by regularizing the corresponding weights, with the regularization strength controlled through smoothing parameters $\lambda_{k,j} \in \mathbb{R}$ for each structured nonlinear term $f_{k,j}$, $k \in \{1, \dots, K\}$, $j \in \{1, \dots, r_k\}$ and appropriate penalty

matrices $\mathbf{S}_{k,j} \in \mathbb{R}^{L_{k,j} \times L_{k,j}}$ (see, e.g., Wood 2017). Optimization of the model is done by minimizing the corresponding penalized negative log-likelihood of $\mathcal{D}$

$$-\log p_{\mathcal{D}}(y|\hat{\boldsymbol{\theta}}(\mathbf{v})) + \sum_{k=1}^K \sum_{j=1}^{r_k} \lambda_{k,j} \mathbf{w}_{k,j}^{\top} \mathbf{S}_{k,j} \mathbf{w}_{k,j}.$$

Tuning a large number of smooth terms, potentially in combination with DNNs, is a challenging task. Nonlinear structured additive models alone require a sophisticated estimation procedure, often based on second-order information and full batch training. This gets even more complicated in semi-structured models. As deep learning platforms allow for custom optimization routines, one possible optimization strategy could be an iterative procedure alternating between the structured and unstructured model parts. This would allow the use of a stochastic gradient descent routine for the DNN part(s), and a classical statistical optimization for the structured model part(s) including the estimation of smoothness parameters. Here however, we propose an alternative strategy that fosters easy training and tuning by defining the smoothness of each effect in terms of the degrees of freedom (df ; see, e.g., Buja, Hastie, and Tibshirani 1989). This approach can be implemented efficiently using the Demmler-Reinsch orthogonalization (see, Ruppert, Wand, and Carroll 2003). The latter can easily be parallelized or sped up using a randomized singular value matrix decomposition (Erichson et al. 2019). We thereby also allow for meaningful default penalization and comparability of effect complexities by setting $df_{k,j}$ to the same value df_k^* for all smooth effects j of the k th parameter. At the same time, we ensure enough flexibility by choosing $df_k^* = \min_{j \in \{1, \dots, r_k\}} \max df_{k,j}$, that is, the largest possible degree of freedom that leaves the least flexible smoothing effect unpenalized and regularizes all others to have the same amount of flexibility. After estimating the smoothing parameters $\lambda_{k,j} \in \mathbb{R}$, optimization can be done efficiently and jointly for all model components using a stochastic gradient descent routine.

4.2. Implementation of the Orthogonalization

The orthogonalization proposed in the previous section can be implemented in different variants, depending on the batch size used in training. When using full-batch gradient descent, the projection $\mathcal{P}_X^{\perp}$ can be calculated upfront and applied in each epoch, yielding exact orthogonalization. For mini-batch training with batch matrices $\mathbf{X}_b$, we calculate the respective projections for each batch using, for example, a QR decomposition $\mathcal{P}_{\mathbf{X}_b}^{\perp} = (\mathbf{I} - \mathbf{Q}_b \mathbf{Q}_b^{\top})$ with $\mathbf{X}_b = \mathbf{Q}_b \mathbf{R}_b$ that allows for stable calculation and inclusion in fully automatic differentiated routines (see, e.g., Roberts and Roberts 2020). In the next section, we will investigate the difference between full-batch gradient descent and its stochastic gradient descent variant using mini-batch orthogonalization.

5. Numerical Experiments

We conduct three different simulation experiments to assess the goodness-of-fit in terms of the structured effect estimation and the prediction performance. Our first experiment demonstrates the efficacy of the orthogonalization cell in practice. In the

second experiment, we examine the properties of the orthogonalization operation when using mini-batch stochastic gradient descent training. The third experiment compares the proposed framework against classical statistical regression frameworks to demonstrate that estimating a penalized SADR model works equally well when cast as a neural network. Unless stated otherwise, the structured nonlinear effects $f_{k,j}$ in all experiments and benchmarks are instantiated using thin-plate regression splines (see, e.g., Wood 2017).

5.1. Identifiability and Interpretability

Here, we mimic a situation where the practitioner's interest explicitly lies in decomposing certain feature effects into structured linear, structured nonlinear and unstructured nonlinear parts, for reasons of interpretability. We simulate 10 dataset replicates with $n = 1500$ observations and $p = 10$ features drawn independently from a uniform distribution $\mathcal{U}(-1, 1)$. For the response Y we consider the cases $Y \sim \mathcal{N}(\eta, s^2)$, $s = 0.1, 1, 10$ (Normal), $y \sim \text{Ber}(\text{sigmoid}(\eta))$ (Bernoulli) and $y \sim \text{Po}(\exp(\eta))$ (Poisson). The predictor η ($K = 1$) contains linear and nonlinear effects of all features, as well as an interaction term of the 10 features:

$$\eta = b + \sum_{j=1}^{10} x_j w_j + \sum_{j=1}^{10} f_j(x_j) + \log_{10}(\prod_{j=1}^{10} (x_j + 2)).$$

The weights $w_1, \dots, w_{10}$ are defined as $\frac{2}{1}, \frac{2}{2}, \dots, \frac{2}{10}$. The functions $f_1, \dots, f_{10}$ are 10 different nonlinear functions defined in the supplementary material B. Our framework explicitly models the true linear and nonlinear terms by separating both effects via orthogonalization. Further, to account for the interaction, an unstructured DNN predictor is defined using a fully-connected network with ReLU activation function (defined as $\tau(x) = \max(0, x)$) and 32 and 16 hidden units in two hidden layers, respectively. By projecting the output of the second-last layer into the orthogonal complement of the structured predictors, we ensure identifiability of the linear and nonlinear effects. We do not fine-tune the model but rather use the Demmler-Reinsch orthogonalization approach described in Section 4.1 and train for a fixed number of 2000 epochs. All models are optimized using the Adam optimizer Kingma and Ba (2014) with a learning rate of 0.01 and a batch size of 32.

Results. Figure 3 visualizes the estimated and true nonlinear relationships between selected features and the response. Overall the resulting estimates in Figure 3 demonstrate the capability of the framework to recover the true partial nonlinear effects, and only the simulation using a normal distribution with $\text{var}(Y) = 100$, which amounts to a maximum of 4% signal-to-noise ratio (predictor variance divided by noise variance), shows an overfitting behavior. Most importantly, the results highlight that the DNN predictor, which is also fed all 10 features, does not learn the linear or nonlinear part of the structured effects, and thereby the structured part of the additive predictor η remains identifiable and interpretable as constructed and desired.

5.2. Mini-Batch Orthogonalization

As discussed in Section 4.2, a full-batch orthogonalization yields an exact projection, while various large-scale applications require mini-batch training, and hence only permit an

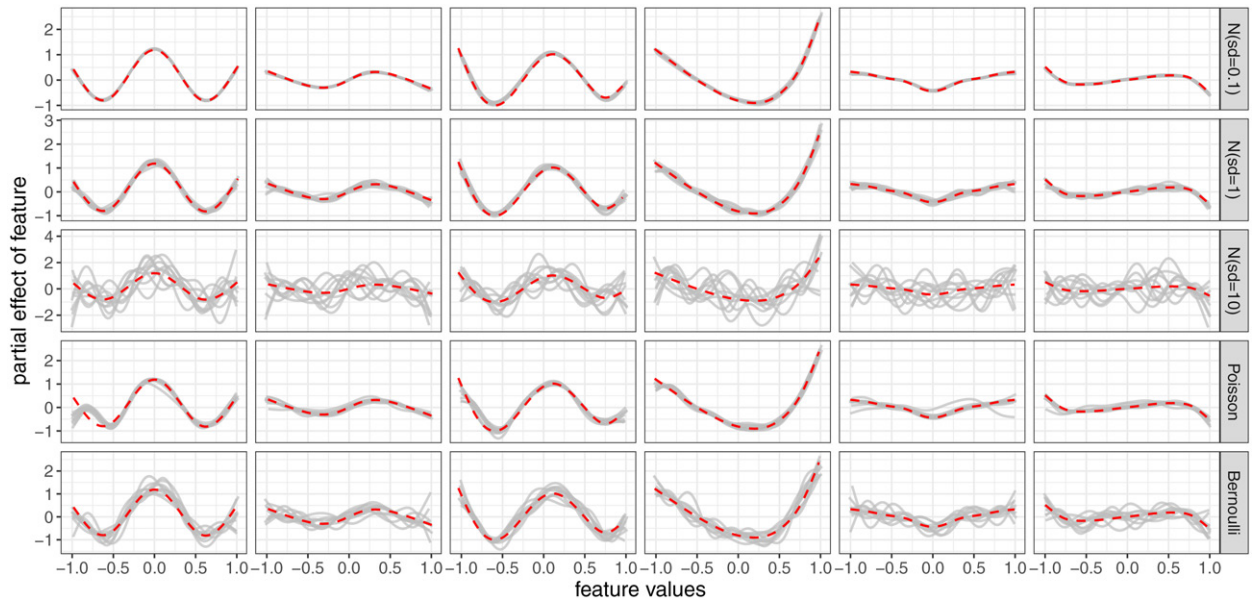


Figure 3. Nonlinear partial effects of six selected features (columns) on the mean of the response from the five different distributions (rows) with true effect in red and estimated functions of the 10 replicates in gray.

approximate orthogonalization. Here, we provide evidence that the latter works equally well in practice. We therefore simulate linear models $Y = Xw + \varepsilon$, where $X \in \mathbb{R}^{n \times p}$ and $\varepsilon \sim \mathcal{N}(\mathbf{0}_n, I_n)$, with different number of samples $n \in \{100, 1000, 10,000\}$ and features of size $p \in \{1, 10\}$ drawn independently from $\mathcal{N}(0, 1)$. We define w to be equally spaced coefficients from -3 to 3 ($w = -3$ for $p = 1$). We then check whether the true effects of these features can be recovered in the presence of a DNN (fully-connected neural network with ReLU activation and 100 and 50 hidden units in two hidden layers). The DNN is provided the same input features as defined for the structured part using either no orthogonalization, full-batch or mini-batch orthogonalization with batch sizes $B \in \{25, 50\}$. The models are trained for 1000 epochs using Adam optimizer with early stopping and a learning rate of 0.001. We repeat each experiment 10 times based on different realizations of standard normal errors ε .

Results. Figure 4 compares the oracle linear model with the SSDR model with and without orthogonalization by looking at the squared deviations of the estimated model coefficients w . Results show no notable difference between training with different batch sizes when using the orthogonalization. The figure also highlights the problem of identifiability when not orthogonalizing, yielding large deviations from the ground truth in all settings except for the ones with a large number of observations ($n = 10,000$) and mini-batch training.

5.3. Model Comparison

In this simulation, we compare the estimation and prediction performance of our framework with three other approaches that implement SADR based on a location, scale and shape parameterization. More specifically, we run our approach against a likelihood-based optimization (gamlss; Rigby and Stasinopoulos 2005), a Bayesian optimization (bamls Umlauf, Klein, and Zeileis 2018) and a model-based boosting routine (MBB; Mayr et al. 2012). For three different distributions (normal, gamma,

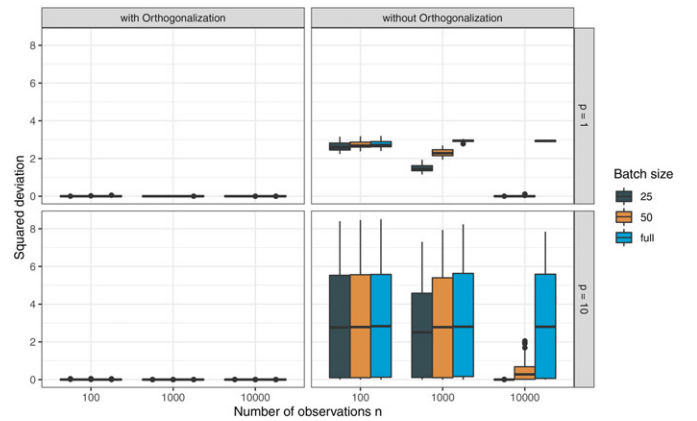


Figure 4. Squared deviations of model coefficients of the oracle and the estimated model (y-axis) for different numbers of observations (x-axis), features (rows), and batch sizes (colors) both with and without orthogonalization (columns). The differences in squared deviations for “with orthogonalization” are on the scale of 10^{-5} and thus negligible.

logistic), we investigate a combination of different sample sizes $n \in \{300, 2500\}$ and different numbers of linear feature effects in the location ($p \in \{10, 75\}$), while fixing the number of linear feature effects in the scale parameter to two. In addition to the linear feature effects, we add 10 nonlinear effects for the location and two nonlinear effects for the scale. Further details can be found in supplementary material B. The SSDR models are trained using Adam with a learning rate of 0.001, batch size of 32, and the number of epochs selected by 5-fold cross-validation. The simulation results are replicated 20 times.

Results. Table 1 summarizes the mean log-scores of 0.25n test data points and mean RMSE measuring the average of the deviations between true and estimated structured effects. We observe that our approach (SSDR) often yields the best or second-best estimation and prediction performance while being robust to more extreme scenarios in which the number of observations is small in comparison to the number of feature effects. In this situation, other approaches tend to suffer from convergence

Table 1. Median and median absolute deviation of the mean negative predictive log-scores and mean RMSE values of estimated weights and nonlinear point estimates across all settings and 20 replications.

	n	p	Negative Log-scores				RMSE			
			bamlss	SSDR	gamlss	MBB	bamlss	SSDR	gamlss	MBB
Normal	300	10	1.51 (0.68)	1.85(0.55)	7.08(8.07)	4.20(0.47)	0.89(0.61)	0.37 (0.28)	0.59(0.15)	0.84(0.22)
		75	> 10e20	2.84 (0.94)	> 10e20	10.4(0.95)	1.05(1.14)	0.47 (0.42)	0.67(0.26)	1.29(0.86)
	2500	10	0.55 (0.06)	0.96(0.18)	0.57(0.08)	3.71(0.24)	0.50(0.71)	0.22 (0.22)	0.25(0.34)	0.66(0.35)
		75	0.64 (0.06)	1.11(0.12)	0.69(0.07)	8.85(0.51)	0.48(0.70)	0.19 (0.22)	0.24(0.32)	1.14(0.64)
Gamma	300	10	1.15(0.10)	1.32(0.31)	1.04 (0.09)	1.13(0.11)	0.13(0.06)	0.14(0.04)	0.08 (0.02)	0.11(0.04)
		75	1.50 (0.36)	2.34(0.87)	2.05(0.99)	1.56(0.15)	0.15(0.06)	0.18(0.07)	0.12 (0.05)	0.14(0.05)
	2500	10	1.01(0.02)	0.93(0.03)	0.83 (0.02)	0.96(0.03)	0.19(0.18)	0.10(0.04)	0.04 (0.03)	0.10(0.08)
		75	1.01(0.03)	1.24(0.05)	0.84 (0.03)	1.01(0.04)	0.19(0.20)	0.07(0.03)	0.04 (0.02)	0.11(0.07)
Logistic	300	10	1.44 (0.14)	1.75(0.12)	4.38(2.84)	3.28(0.37)	1.69(0.98)	0.28 (0.14)	0.61(0.18)	0.97(0.36)
		75	2.55(0.48)	2.22 (0.18)	124(104)	4.89(0.26)	1.82(1.21)	0.26 (0.15)	0.66(0.44)	1.30(0.84)
	2500	10	1.7(0.04)	1.15 (0.06)	1.15(0.04)	> 10e20	2.20(0.82)	0.18 (0.15)	0.24(0.32)	0.73(0.41)
		75	2.47(0.36)	1.16 (0.08)	1.23(0.05)	4.71(0.1)	3.52(0.97)	0.13 (0.13)	0.25(0.32)	1.19(0.66)

NOTE: The best-performing approach is highlighted in bold.

problems. We conclude that the estimation of SADR within our DNN framework works at least as good as classical statistical approaches, but yields more robust results in high-dimensional settings.

6. Benchmark Studies

In addition to the experiments on synthetic data, we provide a number of benchmarks in several real-world datasets in this section. If not stated otherwise, we report measures as averages and standard deviations (in brackets) over 20 random network initializations. Unless reported otherwise, activation functions for DNN predictors in SSDR are ReLU for hidden layers and linear for output layers. In the supplementary material, additional investigations of our framework are provided, checking its competitiveness for quantile regression (supplementary material C.1), calibrated regression (supplementary material C.2), and high-dimensional classification problems (supplementary material C.3).

6.1. Deep Mixed Models

Tran et al. (2020) use a panel dataset with 595 individuals and 4165 observations from Cornwell and Rupert (1988) as an example for fitting deep mixed models by accounting for within-subject correlation. Performance is measured in terms of within-subject predictions of the log of wage for future time points. We follow their analysis by training the model on the years $t = 1, \dots, 5$ and predicting the years $t = 6, 7$. We use a normal distribution with constant variance and model the mean with the same neural network predictor as implemented by Tran et al. (2020). However, instead of being part of the DNN predictor, the subject ID is included as a structured random effect w_i for each individual i : $\log\text{-wage}_{i,t} \sim \mathcal{N}(b + w_i + d_\mu(\mathbf{x}_{i,t}), \exp(d_\sigma(\mathbf{x}_{i,t})))$, with $\mathbf{x}_{i,t}$ being the 12 features also used in Tran et al. (2020), individual-specific random effects $w_i \sim \mathcal{N}(0, \tau^2) \forall i$, and d_μ, d_σ two different fully-connected neural networks with two hidden layers and five neurons each. The model is estimated with default settings of Adam, batch size of 32 and the number of epochs chosen by cross-validation.

Results. Our approach yields an average MSE of 0.04 (0.005) which makes our method competitive with the approach of Tran et al. (2020), who report an MSE of 0.05 for the given data split.

Table 2. Comparison of negative log-scores of different methods (columns) on four different UCI repository datasets (rows).

	DNN	SSDR (OZ)	SSDR	GPR	IR	GPB
Airfoil	5.66(0.12)	3.16 (0.12)	3.35(0.00)	3.17(6.82)	3.29(1.86)	3.21(4.70)
Boston	3.61(0.02)	3.05(0.19)	3.02(0.23)	2.79(2.05)	3.36(5.19)	2.70 (1.91)
Diabetes	5.34(0.02)	5.33 (0.04)	5.50(0.19)	5.35(5.76)	5.71(2.97)	5.33(6.24)
Forest F.	1.75(0.00)	1.85(0.10)	2.01(0.10)	1.75(7.09)	1.00 (1.94)	2.07(9.25)

Bold values highlight the best value possible.

6.2. Deep Calibrated Regression

Next, we use the datasets *Diabetes*, *Boston*, *Airfoil*, and *Forest Fire* analyzed by Song et al. (2019) to benchmark our SSDR approach against the two post-hoc calibration methods isotonic regression (IR; Kuleshov, Fenner, and Ermon 2018) and the GP-Beta model (GPB; Song et al. 2019) with 16 inducing points. The uncalibrated model for the latter two is a Gaussian process regression (GPR) which performed better than ordinary least squares and standard neural networks in Song et al. (2019). SSDR directly models the parameters of a normal distribution. Here, we only fine-tune the specific structure for the predictors $\eta_\mu = \mu, \eta_\sigma = \log(\sigma)$, which consist of structured linear and nonlinear effects, as well as a DNN for all features (see the supplementary material C.2 for details about each model's specification). We further investigate the effect of orthogonalization on the predictive performance (as suggested by an anonymous reviewer) by comparing the SSDR model both with orthogonalization (OZ) and without, and add a vanilla DNN without any structured model part to analyze the effect of the unstructured predictor. We split the data into 75% for training and 25% for model evaluation, measured by negative log-scores.

Results. Table 2 suggests that compared to other calibration techniques our method yields more stable results while permitting interpretable structured effects for features of interest. Even though we did not fine-tune the output distribution, SSDR performs as well as the benchmarks in terms of average negative log-scores. The comparison between the vanilla DNN and SSDR shows that predictive performance can indeed benefit from an additional structured model part, that is, to make linear effects explicit. While orthogonalization might limit the flexibility of the DNN, resulting in a decline in performance (Forest Fire), an SSDR model with orthogonalization sometimes also improves upon the model without the identifiability adjustment (Airfoil,

Table 3. Overview of features in the dataset, their description and how the features are parameterized.

Feature(s)	Description	Effect
$x_1, \dots, x_3$	room types	dummy-effect
$x_4, \dots, x_9$	number of beds	dummy-effect
$x_{10}, \dots, x_{13}$	number of bedrooms	dummy-effect
z_1	accommodates	thin-plate regression splines
z_2	last review (in days)	thin-plate regression splines
z_3	reviews per month	thin-plate regression splines
z_4	review scores	thin-plate regression splines
$z_{5,1}, z_{5,2}$	longitude, latitude	tensor product spline
u_1	images	convolutional neural network
u_2	description	embedding layer + fully-connected DNN
u_3	$x_1, \dots, x_{13}, z_1, \dots, z_4$	fully-connected DNN (16-4-2)

NOTE: Fully connected denotes a fully connected layer with a single unit. The embedding layer is of size 100 for a lookup of 10,000 words with maximum sentence length of 100. The convolutional neural network architecture is described in the supplementary material D.2. The fully connected DNN architecture consists of 16, 4, and 2 fully connected layers with dropout layers in between and is used to model the interactions of structured features.

Diabetes). This is most likely the case when a focus on the structured model part is important.

7. Application to Multimodal Data

Finally, we present an application of our framework to Airbnb price listing data, available at <http://insideairbnb.com/get-the-data.html>. Various data modalities are included in the data, such as numeric variables for latitude and longitude, integer variables such as the number of bedrooms, textual information such as a room description, dates as well as an image of each property (size $200 \times 200 \times 3$), see Table 3. We will focus on apartments in Munich, Germany consisting of 3504 observations and 73 features of mixed data modalities. Our goal is to predict the listing price of each apartment in an interpretable structured

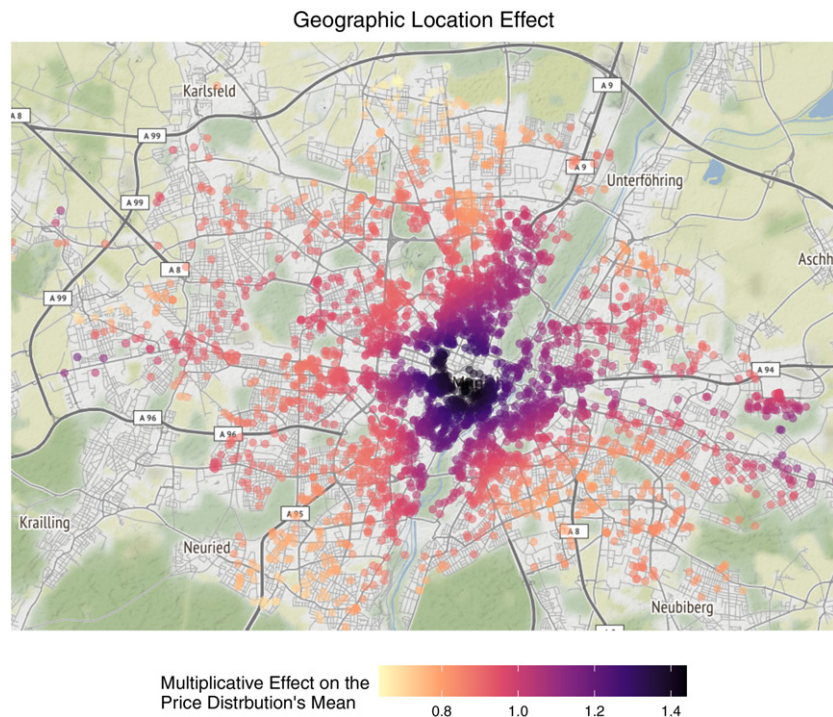
additive model, while also accounting for the room description and images. As the dataset is relatively small and the information content in the images seems also not substantially decisive for the room price, we use this application to demonstrate a further property of the orthogonalization—its regularization effect. To this end, we compare the model using two different specifications. One model is specified with structured and unstructured predictors which are simply added up. The other specification regularizes the learned latent image effects by orthogonalizing them w.r.t. all specified structured features. In other words, the second model subtracts the structured information from the learned image information in order to regularize the DNN. For model inspection and fine-tuning, we set aside 10% of the data for testing and use 10% of the resulting training data for early stopping.

Model Specification We use a log normal distribution for $\mathcal{D}$ which is parameterized by its location (mean) μ and scale parameter σ with corresponding additive predictors η_μ and η_σ , respectively. To model the mean of the logarithmic apartment prices (μ), we define our linear predictor η_μ as

$$\eta_\mu = b_\mu + \sum_{j=1}^{13} x_j w_{\mu,j} + \sum_{j=1}^4 f_{\mu,j}(z_j) + f_{\mu,5}(z_{5,1}, z_{5,2}) + d_{\mu,1}(u_1) + d_{\mu,2}(u_2) + d_{\mu,3}(u_3),$$

with features given in Table 3. For the scale parameter σ we observe a better validation performance when only including a few structured predictors. As for the mean we include the room type as linear (dummy-)effects and a tensor product spline for location. Further model specifications and test results are given in supplementary material D.

Results. Results suggest that structured effects are the main driving factor in our model. As images and descriptions are rather noisy in this dataset, this result is not surprising. We observe a strong spatial effect of the geographic location on the log price of apartments. As shown in Figure 5, the more

**Figure 5.** Estimated multiplicative effect (color) of the apartment's geographic location (points on the map) on the mean price value.

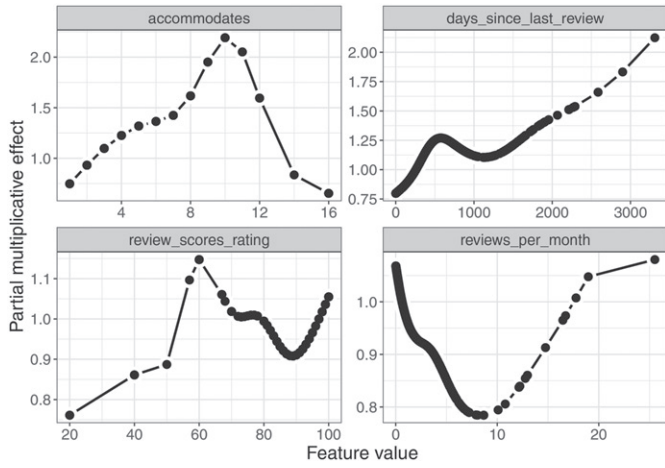


Figure 6. Estimated multiplicative effect (color) of accommodates, days since last review, reviews per month and review rating, on the mean price value.

Table 4. Correlations of predicted and true values on train and test set (columns) for the six different models (rows).

Model	Train	Test
Structured	0.68	0.64
Structured w/ DNN	0.68	0.64
Full w/ OZ	0.84	0.67
Full w/o OZ	0.76	0.63
Structured + DNN + Texts	0.68	0.64
Images only	0.17	0.16

Bold values highlight the best value possible.

expensive apartments are located in the center of Munich, close to the English garden, or in rather prestigious areas. Relative to a typical apartment in Munich, the price for an apartment in the central part of Munich can, for example, be up to 1.4 times more expensive, while a comparable apartment in, for example, the south might cost only 60% of the average room. The partial effect of the latent feature learned via the images and texts is only slightly correlated with the response. The multiplicative effect of remaining structured nonlinear effects on the mean prices are visualized in Figure 6. Here the number of accommodated people has the potentially highest impact with a tent-shaped partial multiplicative effect of over two times the average apartment price at 10 accommodated people. The effects for η_σ can be found in Table 2 and Figure 3 in the supplementary material.

We also compare different model specifications of η_μ based on the correlation of predicted and true price values on train and test set. Table 4 compares the model with full information as given in Table 3 (Full), with and without orthogonalization (w/o OZ), with a model containing only structured effects (Structured), a model with structured effects and one DNN for the tabular data u_3 (Structured + DNN), a model with structured effects, DNN and text embedding (Structured + DNN + Texts) as well as a convolutional neural network where only the images are used (Images only). We here also find that the structured model part contributes the most, but adding the image information can help—however, only when orthogonalized w.r.t. the structured effects.

8. Conclusion and Outlook

In this work, we demonstrate why the combination of structured additive models and DNNs is not straightforward. We

present a solution that provably permits the identification of structured effects in the presence of more flexible DNNs. We further develop a unified network architecture that combines SADR and deep learning by embedding the former into an overarching DNN. Using an orthogonalization cell enables the estimation of (interpretable) structured feature effects next to unstructured DNN predictors while ensuring identifiability of the structured model part(s). Simulations, benchmark studies, and a multimodal learning application demonstrate the generality and robustness of our proposed approach.

While ensuring identifiability and interpretability of structured effects in the additive predictor, we note that the orthogonalization approach is much more versatile and can be applied to various other use cases beyond the identification of structured effects. In particular, using the orthogonalization, a structured effect can be “subtracted” from a latent effect learned from unstructured data input, for example, to adjust for confounders.

As noted by an anonymous reviewer, modeling a mixture of distributions can potentially be more flexible than a single distributional regression. Similar to the embedding of a distribution in a neural network as described in Section 3.2.1, it is also possible to model a mixture of distributions. This has been recently done by Rügamer et al. (2022) extending our approach to a neural network-based implementation of mixtures of experts distributional regression.

Supplementary Materials

Further Details: The Supplementary Material includes proofs, algorithmic details as well as further specifications and results of numerical experiments.

Reproducibility: All codes used for this work are available at https://github.com/davidruegamer/semi-structured_distributional_regression

Acknowledgments

We thank Almond Stöcker and Dominik Thalmeier for their comments and helpful discussions.

Funding

David Rügamer has been partly funded by the German Federal Ministry of Education and Research (BMBF) under grant no. 01IS18036A. Nadja Klein acknowledges support by the Deutsche Forschungsgemeinschaft (DFG; German research foundation) through the Emmy Noether grant KL 3037/1-1.

ORCID

David Rügamer <http://orcid.org/0000-0002-8772-9202>

Nadja Klein <http://orcid.org/0000-0002-5072-5347>

References

- Agarwal, R., Melnick, L., Frosst, N., Zhang, X., Lengerich, B., Caruana, R., and Hinton, G. E. (2021), “Neural Additive Models: Interpretable Machine Learning with Neural Nets,” in *Advances in Neural Information Processing Systems* (Vol. 34), pp. 4699–4711. [89]
- Arora, S., Cohen, N., Hu, W., and Luo, Y. (2019), “Implicit Regularization in Deep Matrix Factorization,” in *Advances in Neural Information Processing Systems*, pp. 7411–7422. [93]

- Bishop, C. M. (1994), "Mixture Density Networks." Neural Computing Research Group Report: NCRG/94/004. Birmingham: Aston University. [88]
- Blei, D. M., Kucukelbir, A., and McAuliffe, J. D. (2017), "Variational Inference: A Review for Statisticians," *Journal of the American Statistical Association*, 112, 859–877. [88]
- Brás-Geraldes, C., Papoila, A., and Xufre, P. (2019), "Generalized Additive Neural Network with Flexible Parametric Link Function: Model Estimation Using Simulated and Real Clinical Data," *Neural Computing and Applications*, 31, 719–736. [89]
- Buja, A., Hastie, T., and Tibshirani, R. (1989), "Linear Smoothers and Additive Models," *The Annals of Statistics*, 17, 453–510. [94]
- Chen, M. L., Dodd, A., Royer, J., Freschi, L., Schito, M., Ezewudo, M., Kohane, I. S., Beam, A., and Farhat, M. (2018), "Deep Learning Predicts Tuberculosis Drug Resistance Status from Whole-Genome Sequencing Data," *BioRxiv*, 275628. [90]
- Cheng, H.-T., Koc, L., Harmsen, J., Shaked, T., Chandra, T., Aradhye, H., Anderson, G., Corrado, G., Chai, W., Ispir, M., Anil, R., Haque, Z., Hong, L., Jain, V., Liu, X., and Shah, H. (2016), "Wide & Deep Learning for Recommender Systems," in *Proceedings of the 1st workshop on Deep Learning for Recommender Systems*, pp. 7–10. ACM. [89,90]
- Cornwell, C., and Rupert, P. (1988), "Efficient Estimation with Panel Data: An Empirical Comparison of Instrumental Variables Estimators," *Journal of Applied Econometrics*, 3, 149–155. [96]
- Cybenko, G. (1989), "Approximation by Superpositions of a Sigmoidal Function," *Mathematics of Control, Signals and Systems*, 2, 303–314. [90]
- Dabney, W., Rowland, M., Bellemare, M., and Munos, R. (2018), "Distributional Reinforcement Learning with Quantile Regression," in *Proceedings of the AAAI Conference on Artificial Intelligence* (Vol. 32). [88]
- Daxberger, E., Kristiadi, A., Immer, A., Eschenhagen, R., Bauer, M., and Hennig, P. (2021), "Laplace Redux – Effortless Bayesian Deep Learning," in *35th Conference on Neural Information Processing Systems (NeurIPS 2021)*. [88]
- De Waal, D. A., and Du Toit, J. V. (2011), "Automation of Generalized Additive Neural Networks for Predictive Data Mining," *Applied Artificial Intelligence*, 25, 380–425. [89]
- Erichson, N. B., Voronin, S., Brunton, S. L., and Kutz, J. N. (2019), "Randomized Matrix Decompositions Using r ," *Journal of Statistical Software*, 89, 1–48. [94]
- Groll, A., Hambuckers, J., Kneib, T., and Umlauf, N. (2019), "Lasso-Type Penalization in the Framework of Generalized Additive Models for Location, Scale and Shape," *Computational Statistics & Data Analysis*, 140, 59–73. [89]
- Hastie, T., and Tibshirani, R. (1986), "Generalized Additive Models," *Statistical Science*, 1, 297–318. [89]
- Hornik, K., Stinchcombe, M., and White, H. (1989), "Multilayer Feedforward Networks are Universal Approximators," *Neural Networks*, 2, 359–366. [91]
- Hubin, A., Storvik, G., and Frommlet, F. (2018), "Deep Bayesian Regression Models." <https://doi.org/10.48550/arXiv.1806.02160> [89]
- Hüllermeier, E., and Waegeman, W. (2021), "Aleatoric and Epistemic Uncertainty in Machine Learning: An Introduction to Concepts and Methods," *Machine Learning*, 110, 457–506. [88]
- Immer, A., Korzepa, M., and Bauer, M. (2021), "Improving Predictions of Bayesian Neural Nets via Local Linearization," in *International Conference on Artificial Intelligence and Statistics*, pp. 703–711. PMLR. [88]
- Kingma, D. P., and Ba, J. (2014), "Adam: A Method for Stochastic Optimization," *arXiv preprint arXiv:1412.6980*. [94]
- Klein, N., Kneib, T., and Lang, S. (2015), "Bayesian Generalized Additive Models for Location, Scale, and Shape for Zero-Inflated and Overdispersed Count Data," *Journal of the American Statistical Association*, 110, 405–419. [90]
- Klein, N., Kneib, T., Lang, S., and Sohn, A. (2015), "Bayesian Structured Additive Distributional Regression with an Application to Regional Income Inequality in Germany," *Annals of Applied Statistics*, 9, 1024–1052. [89]
- Koenker, R. (2005), *Quantile Regression*, Volume Economic Society Monographs, Cambridge: Cambridge University Press. [89]
- Koenker, R., and Bassett Jr., G. (1978), "Regression Quantiles," *Econometrica: Journal of the Econometric Society*, 46, 33–50. [89]
- Kuleshov, V., Fenner, N., and Ermon, S. (2018), "Accurate Uncertainties for Deep Learning Using Calibrated Regression," in *Proceedings of the 35th International Conference on Machine Learning* (Vol. 80), pp. 2796–2804. [96]
- Lakshminarayanan, B., Pritzel, A., and Blundell, C. (2017), "Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles," in *Proceedings of the 31st Conference on Neural Information Processing Systems (NIPS 2017)*. [88]
- Li, R., Reich, B. J., and Bondell, H. D. (2021), "Deep Distribution Regression," *Computational Statistics & Data Analysis*, 159, 107203. [90]
- Maddox, W. J., Izmailov, P., Garipov, T., Vetrov, D. P., and Wilson, A. G. (2019), "A Simple Baseline for Bayesian Uncertainty in Deep Learning," in *Advances in Neural Information Processing Systems* (Vol. 32). [88]
- Mayr, A., Fenske, N., Hofner, B., Kneib, T., and Schmid, M. (2012), "Generalized Additive Models for Location, Scale and Shape for High-Dimensional Data - A Flexible Approach based on Boosting," *Journal of the Royal Statistical Society, Series C*, 61, 403–427. [95]
- Nelder, J. A., and Wedderburn, R. W. (1972), "Generalized Linear Models," *Journal of the Royal Statistical Society, Series A*, 135, 370–384. [89]
- Pölsterl, S., Sarasua, I., Gutiérrez-Becker, B., and Wachinger, C. (2020), "A Wide and Deep Neural Network for Survival Analysis from Anatomical Shape and Tabular Clinical Data," in *Communications in Computer and Information Science*, 453–464. Cham: Springer. [89,90]
- Rigby, R. A., and Stasinopoulos, D. M. (2005), "Generalized Additive Models for Location, Scale and Shape," *Journal of the Royal Statistical Society, Series C*, 54, 507–554. [89,95]
- Roberts, D. A. O., and Roberts, L. R. (2020), "Qr and Lq Decomposition Matrix Backpropagation Algorithms for Square, Wide, and Deep – Real or Complex – Matrices and their Software Implementation." <https://doi.org/10.48550/ARXIV.2009.10071> [94]
- Rügamer, D., Bender, A., Wiegand, S., Racek, D., Bischl, B., Müller, C., and Stachl, C. (2022), "Factorized Structured Regression for Large-Scale Varying Coefficient Models," Accepted at ECML-PKDD 2022. [90]
- Rügamer, D., Brockhaus, S., Gentsch, K., Scherer, K., and Greven, S. (2018), "Boosting Factor-Specific Functional Historical Models for the Detection of Synchronization in Bioelectrical Signals," *Journal of the Royal Statistical Society, Series C*, 67, 621–642. [93]
- Rügamer, D., Pfisterer, F., Bischl, B., and Grün, B. (2022), "Mixture of Experts Distributional Regression: Implementation Using Robust Estimation with Adaptive First-Order Methods," *arXiv:2211.09875*. [98]
- Ruppert, D., Wand, M. P., and Carroll, R. J. (2003), *Semiparametric Regression*, Cambridge and New York: Cambridge University Press. [94]
- Sarle, W. S. (1994), "Neural Networks and Statistical Models," in *Proceedings of the Nineteenth Annual SAS Users Group International Conference*, Citeseer. [89]
- Song, H., Diethe, T., Kull, M., and Flach, P. (2019), "Distribution Calibration for Regression," in *Proceedings of the 36th International Conference on Machine Learning*, Volume 97 of Proceedings of Machine Learning Research, pp. 5897–5906, PMLR. [96]
- Tran, M.-N., Nguyen, N., Nott, D., and Kohn, R. (2020), "Bayesian Deep Net GLM and GLMM," *Journal of Computational and Graphical Statistics*, 29, 97–113. [89,96]
- Umlauf, N., Klein, N., and Zeileis, A. (2018), "Bamlss: Bayesian Additive Models for Location, Scale, and Shape (and Beyond)," *Journal of Computational and Graphical Statistics*, 27, 612–627. [89,95]
- Wood, S. N. (2017), *Generalized Additive Models: An Introduction with R*, Boca Raton, FL: Chapman and Hall/CRC. [89,91,94]